

CHAPITRE 3

TRAITEMENT DE FORMULAIRES (COTE SERVEUR)

I. Champ de texte envoyé avec POST

C'est la méthode à utiliser par défaut. Les données sont transmises de façon non visible du navigateur au serveur. La syntaxe au niveau du formulaire est : **<form action="newFile.php" method="post">**

La syntaxe au niveau du contrôle de formulaire est :

<input type="text" name="nom_de_variable"

Dans le fichier **newFile.php**, les données sont récupérées dans le tableau `$_POST` :

`$_POST["nom_de_variable"]`

II. Champ de texte envoyé avec GET

Cette méthode est à proscrire pour les passages des mots de passe et autres informations 'sensibles', en revanche elle est pratique pour la mise au point puisque l'on voit dans l'URL les valeurs envoyées au serveur. La syntaxe au niveau du formulaire est

<form action="newFile.php" method="get">

La syntaxe au niveau du contrôle est, comme précédemment :

<input type="text" name="nom_de_variable"

Les formulaires GET s'apparentent, au niveau du fonctionnement interne de HTML, au passage de paramètre par URL, car c'est par l'URL que sont transmises les informations du formulaire. Comme dans le passage par l'URL, les données sont à récupérer dans le fichier **newfile.php** dans le tableau `$_GET` avec cette syntaxe : `$_GET['var_a']` et `$_GET['var_b']`

III. Boutons radio

Ce type de contrôle est à utiliser lorsque l'on veut que l'internaute ne puisse choisir

qu'une option parmi plusieurs

Page-1-

La syntaxe au niveau du contrôle est : (peut fonctionner avec un formulaire GET ou POST)

```
<input type='radio' name=monBoutonRadio value='valeur_a'>Label_a <input  
type='radio' name=monBoutonRadio value='valeur_b'>Label_b
```

.....

Autant de lignes que de boutons, à chaque bouton correspondant une valeur 'valeur_x' et un label, mais tous les boutons doivent porter le même nom pour un ensemble de boutons donné. La valeur choisie va se retrouver dans la variable `$_POST['monBoutonRadio']`, qui va contenir la valeur "valeur_x" suivant le bouton qui est coché

IV. Case à cocher multiple

Ce type de contrôle est à utiliser lorsque l'on veut que l'internaute puisse choisir plusieurs cases à cocher à la fois La syntaxe au niveau du contrôle est : (peut fonctionner avec un formulaire GET ou POST)

```
<input type='checkbox' name=$nom_a > label_a  
<input type='checkbox' name=$nom_b > label_b
```

.....

Autant de lignes que de boutons, à chaque bouton correspondant non plus une valeur mais un nom et un label; pas de valeur à affecter dans ce cas. Pour récupérer la variable : `if (isset($_POST['nom_x']))...` Comme dans l'exercice ci-après il sera souvent plus pratique de mettre les noms des variables dans un tableau de façon à pouvoir explorer les valeurs à l'aide d'une boucle (for, while...)

NB: On ne récupère pas des valeurs (comme on l'a vu ci-dessus la propriété 'value' est absente du formulaire dans ce cas) mais des états logiques.

V. Liste de choix

Liste déroulante à choix unique. On ne peut sélectionner qu'un seul élément de la liste
La syntaxe au niveau du contrôle est : (peut fonctionner avec un formulaire GET ou POST)

Page-2-

```
<SELECT NAME=maListe>  
<OPTION value=nom_a>label_a</OPTION>  
<OPTION value=nom_b>label_b</OPTION>  
.....
```

Autant de lignes que d'éléments dans la liste. On voit que l'on a un seul nom On récupère la valeur (nom_x) dans \$_POST['maListe']

VI. Liste à choix multiple

Liste déroulante à choix multiple : l'internaute peut sélectionner plusieurs options dans la liste La syntaxe au niveau du contrôle est : (peut fonctionner avec un formulaire GET ou POST)

```
<SELECT NAME=maListe[] size=7 multiple>  
<OPTION value=valeur_a>label_a</OPTION>  
<OPTION value=valeur_b>label_b</OPTION>  
.....
```

Autant de lignes que d'éléments dans la liste. On récupère la valeur dans le tableau \$_POST['maListe'] Attention : bien mettre les crochets [] à la suite du nom dans la déclaration HTML. On ne récupère pas une valeur mais directement un tableau de valeurs.

VII. Zone de texte

La syntaxe au niveau du contrôle est : (peut fonctionner avec un formulaire GET ou POST)

```
<textarea name='monTexte' cols=20 rows=5>
```

Racontez ici

`</textarea>`

On récupère la valeur dans `$_POST['montexte']`

Page-3-

VIII. Eviter la multiplication des pages

Pour diminuer le nombre de fichiers sur le serveur (peut avoir un impact sur la rapidité : si le nombre de fichiers augmente, le serveur est plus lent), on peut n'utiliser qu'un seul fichier pour le formulaire et le traitement du formulaire; pour réaliser cela il faut remplacer le nom du fichier dans la balise action:

- Soit par le nom du fichier en cours, mais c'est dangereux, si l'on change le nom du fichier, le formulaire ne fonctionnera plus.
- Soit par la variable `$_SERVER[PHP_SELF]` dans laquelle est stockée en permanence le nom du fichier en cours d'exécution.

La syntaxe au niveau du formulaire est donc:

`<form action=<?php echo $_SERVER[PHP_SELF] ; ?> >`

Ceci permet donc de réactiver la page en cours lorsque l'on a cliqué sur le bouton de soumission du formulaire, c'est encore cette page qui va être chargée, il faut réaliser un test en début de page, si les variables du formulaire sont chargées, il ne faut plus afficher le formulaire mais la page suivante.

Exemple:

Un seul fichier source pour saisir et traiter le formulaire (réception des données)

`<?php`

`echo "Saisir ici votre texte : ";`

`echo "<form action=$_SERVER[PHP_SELF] method=post>";`

`echo "<textarea name='myLife' cols=20 rows=4>Votre texte ici</textarea>";`

`echo "<input type='submit' value='valider'></form>"; echo "Voici ce que vous avez saisi :<br/>";`

```
echo "<br/><i>".$_POST['myLife']. "</i>";
```

```
?>
```

Page-4-

IX. Passage de paramètre par l'URL

On peut passer des informations lors de la construction du lien d'une page vers une autre; la syntaxe est la suivante :

```
<?php
echo "<a href=file.php?var_a='$var_1'&var_b='$var_2'>Passer
les variables</a>";
?>
```

Les variables passées de la page actuelle vers la suivante sont **\$var_1** et **\$var_2** Dans la page suivante les valeurs seront à récupérer dans **\$_GET['var_a']** et **\$_GET['var_b']**

A noter le **&** sert pour séparer les paramètres à passer

Exemple sur le passage de paramètres par URL:

Source: EXEMPLE.php	Résultat
<pre><?php ... /*On remarque le ? avant le nom du premier paramètre et le & entre les paramètres suivants */</pre>	Passer les variables
<pre>?></pre>	

Source: TEST.php	Résultat
<pre><?php //affichage des informations reçues echo "Informations reçues :
"; echo "Nom: ".\$_GET['variable_1']."
"; echo "Prénom: ".\$_GET['variable_2']."
"; ?></pre>	Informations reçues : Nom : Dupond Prénom : René

